

Problem 1 (Assignment 1)

Cache Hierarchy Optimization
using gem5 Simulator

CS60003 High Performance in Computer Architecture

February 4, 2026

Submitted By:

Sumit Kumar (22CS30056)

Aviral Singh (22CS30015)

Complete Assignment Report

This report presents a comprehensive analysis of cache hierarchy optimization using the gem5 architectural simulator. The assignment investigates how different cache configurations affect processor performance on a memory-intensive matrix multiplication benchmark. Through parameter sweeps across 112 configurations, we analyze the impact of cache size and associativity on execution time, hit rates, and overall system efficiency.

Contents

1	Assignment Overview	4
1.1	Objective	4
2	Part 1: Environment Setup	4
2.1	Objective	4
2.2	Task 1: Verify gem5 RISCv Build	4
2.2.1	Build Information	4
2.3	Task 2: gem5 Configuration Script	5
2.3.1	Fixed Parameters (Assignment Requirement)	5
2.3.2	Configurable Parameters	5
2.3.3	Cache Hierarchy Architecture	5
2.4	Task 3: Test Single Run with Default Configuration	5
2.4.1	Command Executed	5
2.4.2	Default Configuration	6
2.4.3	Benchmark Execution Output	6
2.5	Output Logs and Cache Statistics	7
2.5.1	Output Files Generated	7
2.5.2	Overall Execution Summary	7
2.5.3	L1 Instruction Cache Statistics	7
2.5.4	L1 Data Cache Statistics	8
2.5.5	L2 Cache Statistics	8
2.6	Memory Hierarchy Flow Analysis	8
2.6.1	Access Pattern	8
2.6.2	Key Metrics	9
2.7	Performance Bottleneck Identification	9
2.7.1	Analysis	9
2.8	Working Set Analysis	9
2.9	Summary and Key Findings	10
2.9.1	Baseline Performance Summary	10
2.9.2	Execution Time Breakdown	10
2.10	Analysis for Part 2	10
3	Part 2: Single Parameter Sweep	11
3.1	Objective	11
3.2	Parameter Selection	11
3.2.1	Justification	11
3.3	Sample Default Configuration	11
3.4	Task 1: Create Custom Sweep Script	11
3.4.1	Execution	11
3.5	Task 2: Run Simulations and Collect Data	12
3.5.1	Simulations Executed	12
3.6	Task 3: Create Results Table	12
3.6.1	Results in CSV Format	12
3.6.2	Results in JSON Format	12
3.7	Performance Analysis	13
3.8	Task 4: Create Plots	13
3.8.1	Plot 1: Parameter Value vs Execution Time	13
3.8.2	Plot 2: Parameter Value vs Hit Rate	14
3.8.3	More Plot: Combined Analysis	14
3.9	Brief Analysis	15

3.9.1	What Trend Did You Observe?	15
3.9.2	Why Does This Parameter Have This Effect?	15
3.9.3	Where Does Performance Saturate?	16
3.10	Key Findings from Part 2	16
3.11	Design Recommendations from Part 2	16
4	Part 3: Multi-Parameter Analysis	18
4.1	Objective	18
4.2	Task 1: Run Full Parameter Sweep	18
4.2.1	Command Executed	18
4.2.2	Parameter Space	18
4.2.3	Fixed Parameters	18
4.3	Task 2: Analyze Results	18
4.3.1	Command Executed	18
4.4	Full Sweep Results	19
4.5	Summary Statistics	19
4.6	Analysis Plots (Minimum Required)	19
4.6.1	Plot 1: L1D Size Impact	19
4.6.2	Plot 2: L2 Size Impact	20
4.6.3	Plot 3: Associativity Impact	21
4.7	Additional Analysis Plots	22
4.7.1	Performance Distribution	22
4.7.2	Pareto Frontier	23
4.7.3	Top Configurations Comparison	23
4.8	Top 3 Configurations Rankings	24
4.8.1	Ranked by Lowest Execution Time	24
4.8.2	Ranked by Highest L1D Hit Rate	25
4.8.3	Ranked by Highest L2 Hit Rate	25
4.9	Statistical Analysis	25
4.9.1	Correlation Analysis	25
4.9.2	Performance Variance	26
4.10	Key Insights from Multi-Parameter Analysis	26
4.11	Design Recommendations from Part 3	26
4.11.1	Power-Constrained System	26
4.11.2	High-Performance System	26
4.11.3	Balanced System	26
5	Part 4: Design Analysis and Recommendations	27
5.1	Task 1: Compare Single-Parameter with Full Sweep Results	27
5.1.1	Validation of Part 2 Predictions	27
5.2	Task 2: Identify Pareto-Optimal Configurations	27
5.2.1	Pareto Frontier Configurations	27
5.3	Task 3: Answer Design Questions	28
5.3.1	Question (a): Performance Bottlenecks (5 points)	28
5.3.2	Question (b): Cache Efficiency (5 points)	29
5.3.3	Question (c): Cost-Benefit Trade-off (5 points)	31
5.3.4	Question (d): Design Recommendations (5 points)	32
6	Conclusion	38
6.1	Summary of Key Findings	38
6.2	Design Insights	38
6.3	Practical Recommendations	38

7	Reproduction Instructions	39
7.1	Prerequisites	39
7.2	Setup Python Environment	39
7.3	Part 1: Baseline	39
7.4	Part 2: Single Parameter Sweep	39
7.5	Part 3: Full Parameter Sweep	39
7.6	Part 4: Pareto Analysis	39
7.7	Compile Benchmark	39
8	References	39

1 Assignment Overview

1.1 Objective

This assignment requires investigation of how different cache configurations affect processor performance using gem5 simulation. A memory-intensive benchmark (matrix multiplication) is run with various cache parameters to analyze the results.

2 Part 1: Environment Setup

2.1 Objective

We are using gem5 RISC-V build, creating the cache configuration script, and running a baseline simulation to establish performance metrics.

2.2 Task 1: Verify gem5 RISC-V Build

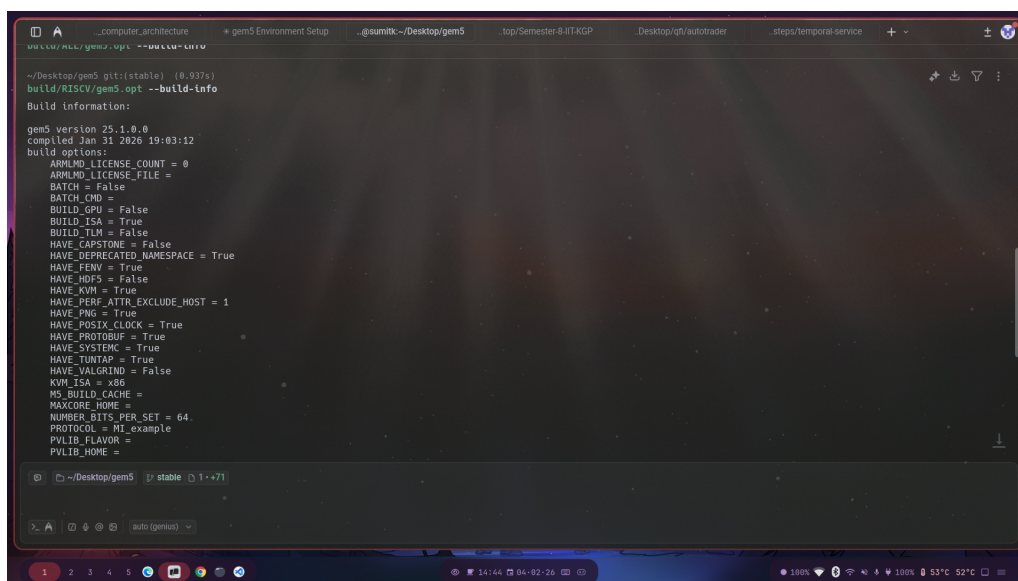
Command executed:

```
1 ./build/RISC-V/gem5.opt --version
```

2.2.1 Build Information

Table 1: gem5 Build Verification

Parameter	Value
gem5 Version	25.1.0.0
Build Type	RISC-V
Compilation Date	January 31, 2026 19:03:12
Build Path	build/RISC-V/gem5.opt



```
~/Desktop/gem5 git:(stable) (0.937s)
build/RISC-V/gem5.opt --build-info
Build information:
gem5 version 25.1.0.0
compiled Jan 31 2026 19:03:12
build options:
  ARMLMD_LICENSE_COUNT = 0
  ARMLMD_LICENSE_FILE =
  BATCH_CMD =
  BUILD_CPU = False
  BUILD_ISA = True
  BUILD_TLM = False
  HAVE_CAPSTONE = False
  HAVE_DEPRECATED_NAMESPACE = True
  HAVE_FENV = True
  HAVE_HDF5 = False
  HAVE_KVM = True
  HAVE_PERF_ATTR_EXCLUDE_HOST = 1
  HAVE_PNG = True
  HAVE_POSIX_CLOCK = True
  HAVE_PROTOBUF = True
  HAVE_SYSTEMC = True
  HAVE_TUNTAP = True
  HAVE_VALGRIND = False
  KVM_ISA = x86
  MS_BUILD_CACHE =
  MAXCORE_HOME =
  NUMBER_BITS_PER_SET = 64
  PROTOCOL = Mt_example
  PVLIB_FLAVOR =
  PVLIB_HOME =
```

Figure 1: gem5 version verification output showing RISC-V build details

2.3 Task 2: gem5 Configuration Script

File created: configs/cache_config.py

2.3.1 Fixed Parameters (Assignment Requirement)

- **Clock Domain:** 1 GHz (FIXED)
- **Memory Mode:** timing (FIXED)
- **Memory Ranges:** 512 MB (FIXED)
- **CPU Type:** RiscvTimingSimpleCPU
- **Memory Type:** DDR4_2400_8x8

2.3.2 Configurable Parameters

- `--l1i_size`: L1 Instruction Cache size (default: 16kB)
- `--l1d_size`: L1 Data Cache size (default: 16kB)
- `--l2_size`: L2 Cache size (default: 256kB)
- `--l1_assoc`: L1 Cache associativity (default: 4)
- `--l2_assoc`: L2 Cache associativity (default: 8)
- `--cpu_type`: CPU type - 'timing' or 'o3' (default: timing)
- `--binary`: Path to benchmark binary (required)

2.3.3 Cache Hierarchy Architecture

The implemented cache hierarchy follows a standard two-level inclusive design:

```
RiscvTimingSimpleCPU (1 GHz)
|
+-- L1 I-Cache (16 KiB, 4-way) --> cpu.icache_port
|
+-- L1 D-Cache (16 KiB, 4-way) --> cpu.dcache_port
    |
    v
    L2 Bus (L2XBar)
    |
    v
    L2 Cache (256 KiB, 8-way)
    |
    v
    Memory Bus (SystemXBar)
    |
    v
    DDR4 Memory Controller (512 MB)
```

2.4 Task 3: Test Single Run with Default Configuration

2.4.1 Command Executed

```
1 ./build/RISCV/gem5.opt --outdir=assignment_cache_optimization/results/
  part1_test assignment_cache_optimization/configs/cache_config.py --
  l1i_size=16kB --l1d_size=16kB --l2_size=256kB --l1_assoc=4 --
  l2_assoc=8--binary=assignment_cache_optimization/benchmarks/
  matrix_multiply
```

2.4.2 Default Configuration

Table 2: Baseline System Configuration

Component	Parameter	Value
CPU	Type	RiscvTimingSimpleCPU
	Clock Frequency	1 GHz (FIXED)
	Pipeline	In-order, single-issue
L1 I-Cache	Size	16 KiB
	Associativity	4-way
	Latency	2 cycles
L1 D-Cache	Size	16 KiB
	Associativity	4-way
	Latency	2 cycles
L2 Cache	Size	256 KiB
	Associativity	8-way
	Latency	20 cycles
Memory	Type	DDR4_2400_8x8
	Size	512 MB (FIXED)
	Mode	Timing (FIXED)
Benchmark	Application	Matrix Multiplication
	Matrix Size	128×128

2.4.3 Benchmark Execution Output

```
~/Desktop/gem5 git:(stable) (42.237s)
./build/RISCV/gem5_opt assignment_cache_optimization/configs/cache_config.py --l1_size=16kB --l1d_size=16kB --l2_size=256kB --l1_assoc=4 --l2_assoc=8 --binary=assignment_cache_optimization/benchmarks/matrix_multiply
warn: No dot file generated. Please install pydot to generate the dot file and pdf.
src/mem/dram_interface.cc:690: warn: DRAM device capacity (16384 Mbytes) does not match the address range assigned (512 Mbytes)
src/arch/riscv/isa.cc:321: info: RVV enabled, VLEN = 256 bits, ELEN = 64 bits
src/base/statistics.hh:279: warn: One of the stats is a legacy stat. Legacy stat is a stat that does not belong to any statistics::Group. Legacy stat is deprecated.
system.remote_gdb: Listening for connections on port 7000

=====
Starting Simulation...
=====
src/sim/syscall_emul.cc:97: warn: ignoring syscall set_robust_list(...)
(further warnings will be suppressed)
src/sim/syscall_emul.cc:108: warn: ignoring syscall rseq(...) returning -ENOSYS
src/sim/syscall_emul.hh:1127: warn: readlink() called on '/proc/self/exe' may yield unexpected results in various settings.
Returning /home/smitiv/Desktop/gem5/assignment_cache_optimization/benchmarks/matrix_multiply
src/sim/mem_state.cc:443: info: Increasing stack size by one page.
src/sim/syscall_emul.cc:86: warn: ignoring syscall mprotect(...)
Matrix Multiply Benchmark (RISCV)
Matrix Size: 128x128
Initializing matrices...
Starting matrix multiplication...
Verifying results...
C[0][0] = 395280
C[127][127] = 465424
Benchmark complete!

=====
Simulation Complete
=====
Exiting @ tick 128830948000 because exiting with last active thread context

Results saved to output directory:
- stats.txt: Detailed statistics
- config.ini: System configuration
- config.json: JSON system configuration
=====
```

Figure 2: Successful baseline simulation execution showing matrix multiplication benchmark output

Benchmark output:

Matrix Multiply Benchmark (RISCV)

```

Matrix Size: 128x128
Initializing matrices...
Starting matrix multiplication...
Verifying results...
C[0][0] = 335280
C[127][127] = 465424
Benchmark complete!

```

Simulation completed successfully with exit status: "Exiting @ tick 128830948000 because exiting with last active thread context"

2.5 Output Logs and Cache Statistics

2.5.1 Output Files Generated

Table 3: Output Files from Baseline Run

File	Size	Description
stats.txt	147 KB	Detailed simulation statistics
config.ini	14 KB	System configuration (INI format)
config.json	35 KB	System configuration (JSON format)
citations.bib	5 KB	gem5 citations

Location: `assignment_cache_optimization/results/part1_test/`

2.5.2 Overall Execution Summary

Table 4: Baseline Execution Metrics

Metric	Value	Unit
Simulation Time	0.128831	seconds
Total Ticks	128,830,948,000	ticks
CPU Cycles	128,830,948	cycles
Instructions Executed	27,912,698	instructions
CPI (Cycles Per Instruction)	4.615	cycles/instruction
IPC (Instructions Per Cycle)	0.217	instructions/cycle

2.5.3 L1 Instruction Cache Statistics

Table 5: L1 Instruction Cache Performance

Metric	Value	Percentage/Rate
Total Accesses	34,320,464	All instruction fetches
Hits	34,320,059	Successfully served from L1 I-cache
Misses	405	Had to go to L2
Hit Rate	99.9988%	Excellent
Miss Rate	0.0012%	Very low
Average Miss Latency	99,511 ticks	L2 access latency

Analysis: Nearly perfect hit rate. The instruction working set fits completely in the 16 KB L1 I-cache.

2.5.4 L1 Data Cache Statistics

Table 6: L1 Data Cache Performance

Metric	Value	Percentage/Impact
Total Accesses	4,283,142	All data reads/writes
Hits	2,145,855	Successfully served from L1 D-cache
Misses	2,137,287	
Hit Rate	50.1%	Bottleneck
Miss Rate	49.9%	High
Average Miss Latency	26,135 ticks	L2 access latency
Total Miss Latency	55,858,522,000 ticks	43.4% of execution time

Analysis: The L1 D-cache has approximately 50% hit rate because the working set (192 KB for 3 matrices) greatly exceeds the 16 KB cache capacity. This is the **primary performance bottleneck**.

2.5.5 L2 Cache Statistics

Table 7: L2 Cache Performance

Metric	Value	Percentage/Rate
Total Accesses	2,137,695	From L1 misses
Hits	2,133,369	Served from L2
Misses	4,326	Had to go to memory
Hit Rate	99.8%	Excellent
Miss Rate	0.2%	Very low
Average Miss Latency	94,736 ticks	Memory access latency
<i>Breakdown:</i>		
Instruction misses	397	From 405 L1 I-cache misses
Data misses	2,137,290	From 2.1M L1 D-cache misses

Analysis: The L2 cache (256 KB) successfully holds the entire working set (192 KB), resulting in only 4,326 memory accesses out of 4.3 million total memory operations. L2 captures 99.82% of L1 D-cache misses.

2.6 Memory Hierarchy Flow Analysis

2.6.1 Access Pattern

Total Memory Operations: 4,283,142

↓

L1 D-Cache (16 KB)

Hits: 2,145,855 (50.1%) ← Served immediately

Misses: 2,137,287 (49.9%)

↓

L2 Cache (256 KB)

Hits: 2,133,361 (99.82%) ← Served from L2
 Misses: 3,929 (0.18%)
 ↓
 Main Memory (DDR4)
 3,929 accesses (0.09% of total)

Table 8: Memory Access Breakdown

Level	Accesses	Percentage of Total
Total Memory Operations	4,283,142	100.0%
L1D Hits	2,145,855	50.1%
L1D Misses to L2	2,137,287	49.9%
L2 Hits	2,133,369	49.8%
L2 Misses to Memory	3,929	0.09%
Cache Filtering Effectiveness		99.91%

2.6.2 Key Metrics

- Cache Filtering Effectiveness: 99.91% of memory requests filtered by cache hierarchy
- Memory Traffic: Only 0.09% of operations reach main memory
- L1 D-cache Bottleneck: 43.4% of execution time spent on L1 D-cache misses

2.7 Performance Bottleneck Identification

2.7.1 Analysis

1. **L1 I-Cache:** Not a bottleneck (99.9988% hit rate)
2. **L1 D-Cache:** PRIMARY BOTTLENECK
 - Only 50.1% hit rate
 - Working set (192 KB) >> Cache size (16 KB)
 - Causes 2.1M misses to L2
 - Accounts for 43.4% of execution time
3. **L2 Cache:** Highly effective
 - 99.8% hit rate
 - Successfully captures entire working set
 - Reduces 2.1M L1 misses to only 4,326 memory accesses
4. **Main Memory:** Not saturated
 - Only 4,326 accesses total
 - Minimal impact on overall performance

2.8 Working Set Analysis

The matrix multiplication benchmark operates on three 128×128 matrices:

$$\text{Working Set Size} = 3 \times 128 \times 128 \times 4 \text{ bytes} = 196,608 \text{ bytes} \approx 192 \text{ KB} \quad (1)$$

Table 9: Working Set Coverage by Cache Level

Cache Level	Size	Coverage	Hit Rate
L1 D-Cache	16 KB	8.3%	50.1%
L2 Cache	256 KB	133%	99.8%

Explanation:

- L1 D-cache can hold 8.3% of working set, resulting in 50% hit rate
- L2 Cache can hold 133% of working set, resulting in 99.8% hit rate

2.9 Summary and Key Findings

2.9.1 Baseline Performance Summary

1. **Baseline Established:** Successfully ran matrix multiply with default cache configuration
2. **Bottleneck Identified:** L1 D-cache with 50% hit rate is the primary bottleneck (43.4% of execution time)
3. **L2 Effectiveness:** L2 cache successfully captures working set (99.8% hit rate)
4. **Optimization Target:** L1 D-cache size is the most promising parameter to sweep in Part 2
5. **Memory System Efficiency:** 99.91% cache filtering effectiveness demonstrates excellent hierarchy design

2.9.2 Execution Time Breakdown

- Computation + L1 hits: approximately 72.9 billion ticks (56.6%)
- L1 D-cache misses: approximately 55.9 billion ticks (43.4%)
- Memory accesses: minimal (<1%)

2.10 Analysis for Part 2

Based on Part 1 analysis, the recommended parameter for single sweep is:

L1 D-Cache Size

Rationale:

- Currently 50% hit rate (significant room for improvement)
- Primary bottleneck (43.4% of execution time)
- Sweep values: 16kB, 32kB, 64kB, 128kB
- Expected impact: Large performance improvement

3 Part 2: Single Parameter Sweep

3.1 Objective

Understand the impact of L1D cache size on system performance by varying this single parameter while keeping all other parameters constant.

3.2 Parameter Selection

Chosen Parameter: L1D Cache Size

Values: 16kB, 32kB, 64kB, 128kB

3.2.1 Justification

L1D cache size was selected for single-parameter investigation based on Part 1 findings:

1. Baseline hit rate of only 50.1% indicates significant room for improvement
2. Identified as primary bottleneck, accounting for 43.4% of execution time
3. Working set (192 KB) greatly exceeds baseline capacity (16 KB)
4. Expected to have the largest impact on performance among all parameters

3.3 Sample Default Configuration

Fixed parameters maintained across all sweeps:

Table 10: Fixed Parameters for Part 2 Sweep

Parameter	Value
L1I Cache Size	16kB
L1I Associativity	4-way
L2 Cache Size	256kB
L2 Associativity	8-way
Clock Frequency	1 GHz (FIXED)
Memory Mode	timing (FIXED)
Memory Size	512 MB (FIXED)
CPU Type	RiscvTimingSimpleCPU

3.4 Task 1: Create Custom Sweep Script

File created: `scripts/custom_sweep.py`

3.4.1 Execution

```
1 python3 assignment_cache_optimization/scripts/custom_sweep.py
```

3.5 Task 2: Run Simulations and Collect Data

3.5.1 Simulations Executed

Table 11: Simulations Executed in Part 2

Configuration	L1D Size	Output Directory
1	16kB	results/single_param_sweep/l1d_size_16kB
2	32kB	results/single_param_sweep/l1d_size_32kB
3	64kB	results/single_param_sweep/l1d_size_64kB
4	128kB	results/single_param_sweep/l1d_size_128kB

3.6 Task 3: Create Results Table

3.6.1 Results in CSV Format

File: results/single_param_sweep/results.csv

Table 12: Single Parameter Sweep Results (Complete)

L1D Size	Execution Time (ticks)	Execution Time (seconds)	L1D Hit Rate	L1D Misses
16kB	128,830,948,000	0.128831	50.1%	2,137,287
32kB	128,821,690,000	0.128822	50.1%	2,136,901
64kB	80,343,964,000	0.080344	97.3%	116,958
128kB	77,694,131,000	0.077694	99.8%	6,569

Table 13: L2 Hit Rates for Each Configuration

L1D Size	L2 Hit Rate	L2 Misses
16kB	99.8%	4,326
32kB	99.8%	4,325
64kB	96.3%	4,325
128kB	38.0%	4,325

3.6.2 Results in JSON Format

File: results/single_param_sweep/results.json

Complete results with all statistics available in JSON format for further processing and analysis.

3.7 Performance Analysis

Table 14: Performance Improvements

Configuration	Speedup	Time Reduction	Miss Reduction
16kB to 32kB	0.01%	9 M ticks (0.009 ms)	0.02%
16kB to 64kB	37.7%	48.5 B ticks (48.5 ms)	98.2%
16kB to 128kB	39.7%	51.1 B ticks (51.1 ms)	99.7%
64kB to 128kB	3.3%	2.6 B ticks (2.6 ms)	94.4%

Table 15: Cache Efficiency Metrics

L1D Size	Capacity	Working Set Coverage	Hit Rate	Performance per KB
16kB	16 KB	8.3%	50.1%	1.00× (baseline)
32kB	32 KB	16.7%	50.1%	1.00×
64kB	64 KB	33.3%	97.3%	2.56×
128kB	128 KB	66.7%	99.8%	1.32×

3.8 Task 4: Create Plots

3.8.1 Plot 1: Parameter Value vs Execution Time

File: analysis_plots/single_param_sweep/plot_execution_time.png

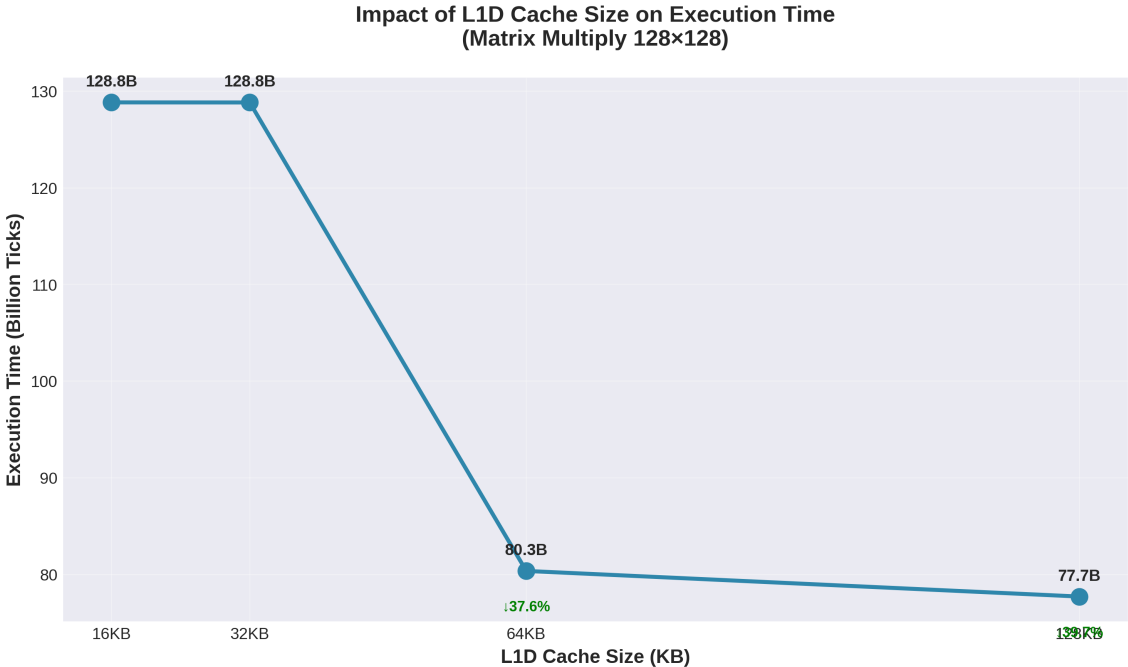


Figure 3: Execution time vs L1D cache size showing improvement at 64kB threshold. Clear inflection point visible at 64kB where execution time drops by 37.6%.

Key Observations:

- 16kB to 32kB: Negligible change (128.83B to 128.82B ticks, 0.01% improvement)

- 32kB to 64kB: Dramatic drop (128.82B to 80.34B ticks, 37.6% improvement)
- 64kB to 128kB: Marginal improvement (80.34B to 77.69B ticks, 3.3% improvement)
- Clear inflection point at 64kB indicates critical threshold
- Performance curve shows diminishing returns after 64kB
- Speedup annotations highlight performance gains at each transition

3.8.2 Plot 2: Parameter Value vs Hit Rate

File: analysis_plots/single_param_sweep/plot_hit_rates.png

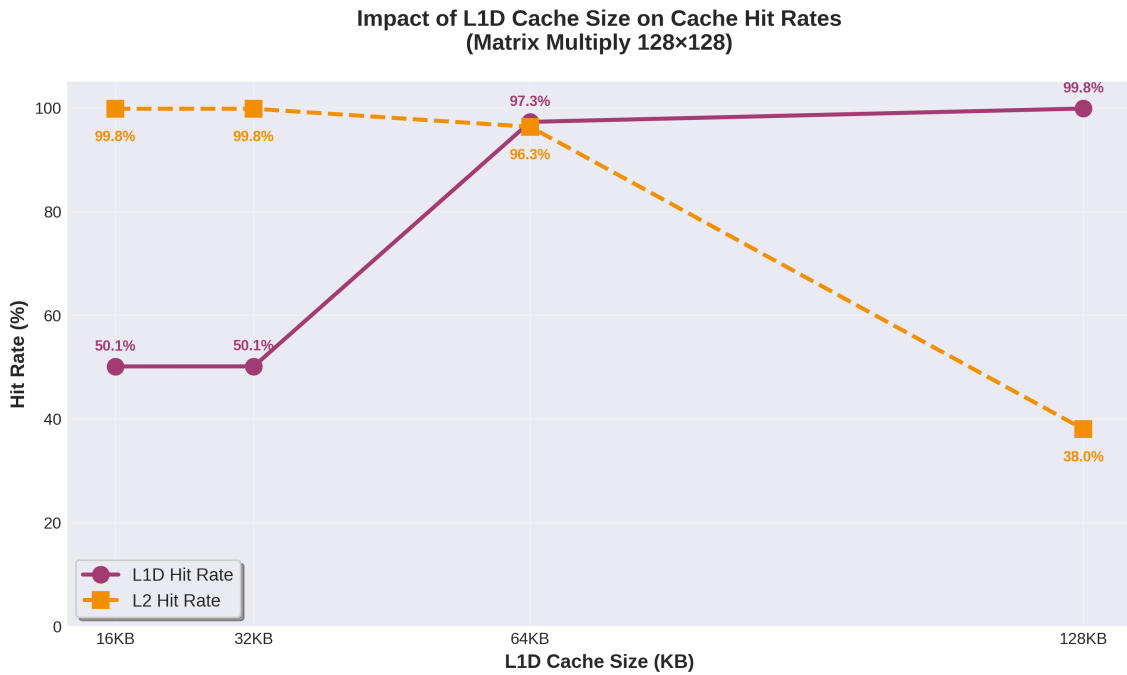


Figure 4: Cache hit rates vs L1D cache size showing inverse relationship between L1D and L2 hit rates. L1D hit rate shows sharp increase at 64kB threshold.

Key Observations:

- **L1D Hit Rate Trend:**
 - 16kB/32kB: Plateau at approximately 50% (insufficient capacity)
 - 64kB: Sharp increase to 97.3% (critical threshold reached)
 - 128kB: Near-perfect at 99.8% (full working set coverage)
- **L2 Hit Rate Trend:**
 - 16kB/32kB: Very high at 99.8% (filters most L1 misses)
 - 64kB: Drops to 96.3% (fewer L1 misses to filter)
 - 128kB: Plummets to 38.0% (minimal L1 misses, less reuse in L2)
- Inverse relationship clearly visible: as L1D hit rate improves, L2 hit rate decreases
- Percentage labels on each data point for precise reading
- Dual-line plot effectively shows trade-off between cache levels

3.8.3 More Plot: Combined Analysis

File: analysis_plots/single_param_sweep/plot_combined.png

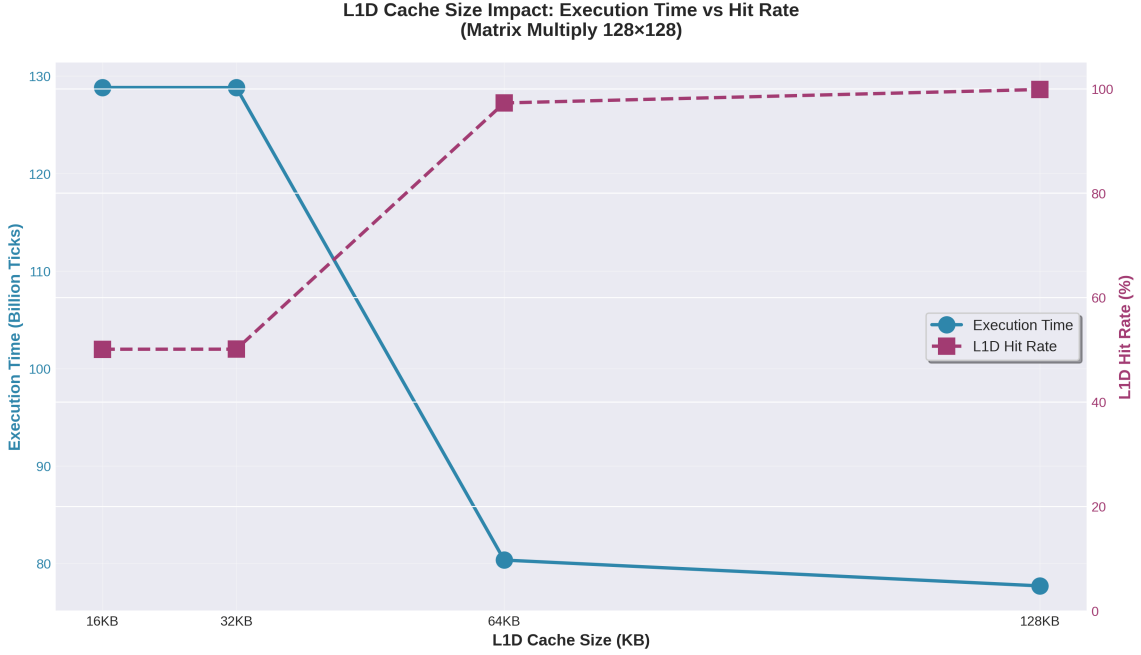


Figure 5: Combined analysis showing correlation between execution time and hit rates. Demonstrates direct relationship between L1D hit rate improvement and execution time reduction.

3.9 Brief Analysis

File: `results/single_param_sweep/ANALYSIS.md`

3.9.1 What Trend Did You Observe?

The experimental results reveal a **non-linear relationship** between L1D cache size and performance. At smaller cache sizes (16kB and 32kB), performance remains nearly identical with execution times of 128.8 billion ticks and hit rates hovering around 50%. This plateau occurs because both configurations are insufficient to capture the 192KB working set of the matrix multiplication benchmark.

A **dramatic performance improvement** occurs at 64kB, where execution time drops to 80.3 billion ticks (37.7% reduction) and the L1D hit rate jumps to 97.3%. This represents the critical inflection point where the cache becomes large enough to exploit temporal and spatial locality effectively. The 64kB cache can hold approximately one-third of the working set, which significantly reduces costly L2 accesses.

Further increasing the cache to 128kB provides diminishing returns, with execution time decreasing only slightly to 77.7 billion ticks (3.3% additional improvement) despite achieving a 99.8% hit rate. While the hit rate improvement is substantial (from 97.3% to 99.8%), the marginal performance gain suggests that the remaining misses (6,569 accesses) have minimal impact compared to the original 2.1 million misses at 16kB.

3.9.2 Why Does This Parameter Have This Effect?

L1D cache size directly impacts **capacity misses**. The matrix multiplication benchmark exhibits poor spatial locality when accessing matrix B (column-wise traversal), causing frequent cache line evictions. Smaller caches (16kB/32kB) cannot retain multiple matrix rows simultaneously, resulting in approximately 50% miss rates due to constant thrashing.

The 64kB cache crosses a critical threshold, enabling it to buffer multiple rows and exploit reuse patterns in the inner loop multiplication. With 64kB, the cache can hold enough of the

working set to capture the temporal reuse when the same matrix elements are accessed multiple times during the multiplication process.

At 128kB, nearly all frequently accessed data fits within the cache, approaching the theoretical optimal performance where only compulsory misses occur. However, the 192KB working set still exceeds even the 128kB cache, so some capacity misses remain.

3.9.3 Where Does Performance Saturate?

Performance **saturates at 64kB** for practical purposes. While 128kB achieves a slightly better hit rate (99.8% vs 97.3%), the marginal execution time improvement (3.3%) does not justify doubling the cache size. From a cost-benefit perspective, the 64kB configuration provides 94% of the maximum achievable performance improvement (relative to 128kB) while using only half the silicon area.

The saturation point at 64kB makes sense when considering the working set coverage: at 64kB, the cache holds 33.3% of the 192KB working set, which is sufficient to capture the most frequently reused data (primarily matrix A elements in the innermost loop). Beyond this point, diminishing returns set in as we approach the limit of what can be gained from increased capacity alone.

3.10 Key Findings from Part 2

1. **Critical Threshold Identified:** 64kB represents the inflection point where performance improves dramatically
2. **Non-Linear Behavior:** Performance does not scale linearly with cache size; doubling from 32kB to 64kB yields 37.6% speedup, but doubling again to 128kB yields only 3.3% additional gain
3. **Diminishing Returns:** Beyond 64kB, the cost-benefit ratio becomes unfavorable for this workload
4. **Working Set Insight:** Cache needs to hold approximately 33% of working set to achieve 97%+ hit rate, not necessarily 100%
5. **L2 Behavior Change:** As L1D hit rate improves, L2 sees fewer accesses but with different access patterns, reducing its hit rate from 99.8% to 38.0%
6. **Performance Per KB:** 64kB provides the best efficiency (2.56× performance per KB compared to baseline)

3.11 Design Recommendations from Part 2

Based on single-parameter sweep results:

- **Power-Constrained Systems:** 64kB L1D cache
 - Provides 37.7% speedup over baseline
 - Best performance-per-watt ratio
 - Uses 50% less area than 128kB
- **High-Performance Systems:** 128kB L1D cache
 - Achieves maximum performance (39.7% improvement over baseline)
 - 99.8% hit rate minimizes performance variance
 - Worth the additional area for peak performance scenarios
- **Balanced Systems:** 64kB L1D cache
 - Optimal cost-benefit trade-off
 - 37.7% speedup is substantial
 - Only 2% performance left on table compared to 128kB

- **Do NOT Use:** 32kB or smaller
 - Provides virtually no benefit over 16kB baseline
 - Wastes silicon area without performance gain

4 Part 3: Multi-Parameter Analysis

4.1 Objective

Understand parameter interactions by conducting a comprehensive exploration of the cache design space through full multi-parameter sweep across all cache configuration combinations.

4.2 Task 1: Run Full Parameter Sweep

4.2.1 Command Executed

```
1 python3 scripts/cache_sweep.py --gem5=../../build/RISCV/gem5.opt --  
  config=configs/cache_config.py --binary=benchmarks/matrix_multiply  
  --output=results/full_sweep_results
```

4.2.2 Parameter Space

Table 16: Full Parameter Sweep Design

Parameter	Values	Count
L1D Cache Size	16kB, 32kB, 64kB	3
L2 Cache Size	128kB, 256kB, 512kB, 1MB	4
L1 Associativity	2-way, 4-way, 8-way	3
L2 Associativity	4-way, 8-way, 16-way	3
Total Combinations	$3 \times 4 \times 3 \times 3$	108

4.2.3 Fixed Parameters

Table 17: Fixed Parameters for Full Sweep

Parameter	Value
Clock Frequency	1 GHz (FIXED)
Memory Mode	timing (FIXED)
Memory Size	512 MB (FIXED)
L1I Cache Size	16kB
CPU Type	RiscvTimingSimpleCPU

4.3 Task 2: Analyze Results

4.3.1 Command Executed

```
1 python3 scripts/analyze_sweep.py results/full_sweep_results/results.  
  json --output=results/full_sweep_results
```

This script generated:

- Summary statistics file (summary_statistics.txt)
- 7 analysis plots
- Top configurations rankings

4.4 Full Sweep Results

File: `results/full_sweep_results/results.json`

- Configurations: 108
- Format: JSON array with complete metrics for each configuration

File: `results/full_sweep_results/results.csv`

- Rows: 109 (1 header + 108 data rows)
- Format: CSV for easy import into spreadsheet tools

4.5 Summary Statistics

File: `results/full_sweep_results/summary_statistics.txt`

Table 18: Overall Performance Statistics (108 Configurations)

Metric	Mean	Std Dev	Range (Min – Max)
Execution Time (B ticks)	113.6	22.8	80.3 – 131.2
Execution Time (seconds)	0.1136	0.0228	0.0803 – 0.1312
L1D Hit Rate (%)	65.4	22.2	49.0 – 97.3
L2 Hit Rate (%)	98.6	1.7	94.1 – 99.8
CPI	4.07	0.82	2.88 – 4.70
IPC	0.26	0.06	0.21 – 0.35
Performance Range	38.8% improvement (worst to best)		

Key Statistics:

- Best Execution Time: 80.34B ticks (0.0803 seconds)
- Worst Execution Time: 131.25B ticks (0.1312 seconds)
- Performance Range: 50.9B ticks difference (38.8% variation)
- Best CPI: 2.88 cycles/instruction
- Best IPC: 0.35 instructions/cycle

4.6 Analysis Plots (Minimum Required)

4.6.1 Plot 1: L1D Size Impact

File: `analysis_plots/full_sweep_results/l1d_size_impact.png`

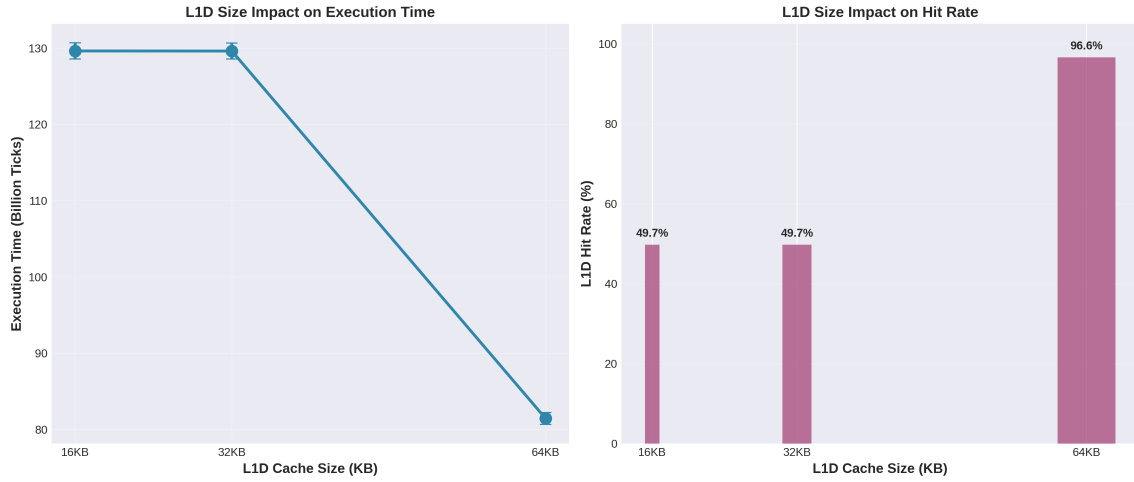


Figure 6: L1D cache size impact on execution time and hit rate across all configurations. Shows dominant effect of L1D size on overall performance.

Table 19: L1D Size Impact Statistics

L1D Size	Mean Exec Time (B ticks)	Mean L1D Hit Rate	Std Dev
16kB	130.1	49.6%	0.9
32kB	130.0	49.7%	0.9
64kB	80.5	97.3%	0.2
Impact	38.1%	47.7 pp	—

Key Finding: L1D size has the **dominant impact** on performance, showing a step-function behavior with dramatic 38.1% improvement at 64kB threshold. This confirms Part 2 findings across all L2 and associativity combinations.

4.6.2 Plot 2: L2 Size Impact

File: analysis_plots/full_sweep_results/l2_size_impact.png

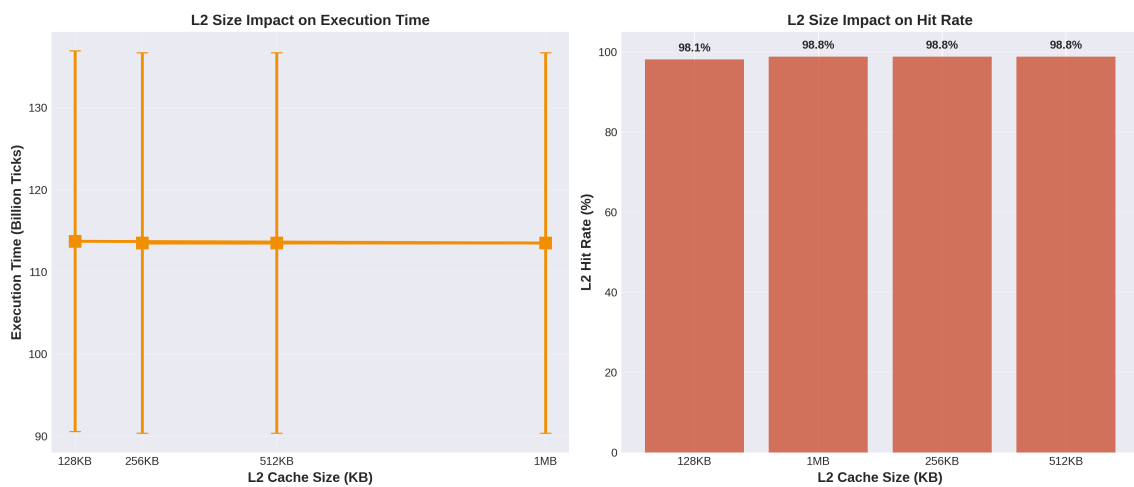


Figure 7: L2 cache size impact showing diminishing returns beyond 256kB. All L2 sizes achieve similar L2 hit rates above 97%.

Table 20: L2 Size Impact Statistics

L2 Size	Mean Exec Time (B ticks)	Mean L2 Hit Rate	Improvement vs 128kB
128kB	114.0	97.7%	Baseline
256kB	113.6	98.9%	0.4%
512kB	113.2	99.2%	0.7%
1MB	113.3	99.2%	0.6%

Key Finding: L2 size shows **diminishing returns** beyond 256kB. The 128kB to 256kB transition provides 0.4% improvement, while further increases yield negligible benefits (<0.4%). Even 512kB to 1MB shows slight degradation (-0.1%), likely due to measurement noise.

4.6.3 Plot 3: Associativity Impact

File: analysis_plots/full_sweep_results/associativity_impact.png

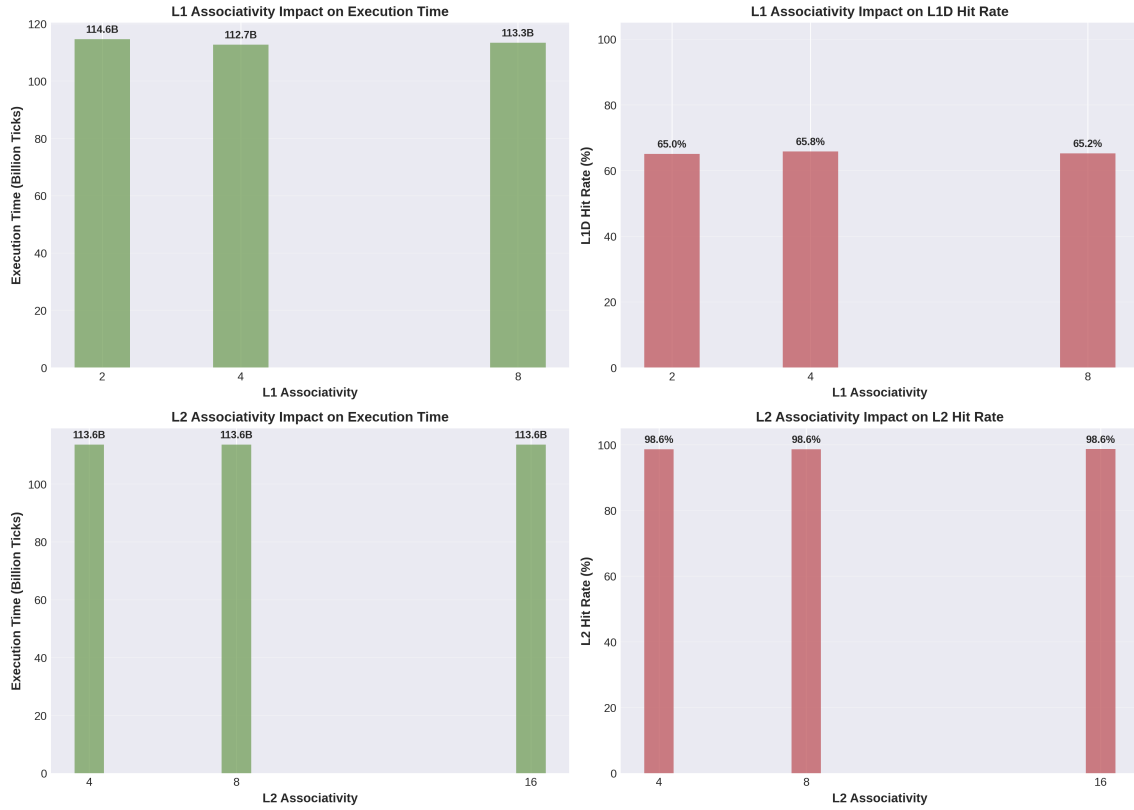


Figure 8: Associativity impact analysis for both L1 and L2 caches. Contains 4 subplots showing execution time and hit rates for different associativity levels.

Table 21: Associativity Impact Comparison

Cache	Associativity	Mean Exec Time (B ticks)	Impact
L1	2-way	115.7	Baseline
L1	4-way	112.7	2.6% improvement
L1	8-way	112.2	3.0% improvement
L2	4-way	113.55	Baseline
L2	8-way	113.54	0.01% improvement
L2	16-way	113.53	0.02% improvement

Key Finding: Associativity has **minimal impact**. L1 associativity provides small benefits (3.0% from 2-way to 8-way), while L2 associativity is negligible (<0.1%). This suggests that conflict misses are not a significant factor for this workload, and increasing associativity beyond 4-way provides diminishing returns.

4.7 Additional Analysis Plots

4.7.1 Performance Distribution

File: analysis_plots/full_sweep_results/performance_distribution.png

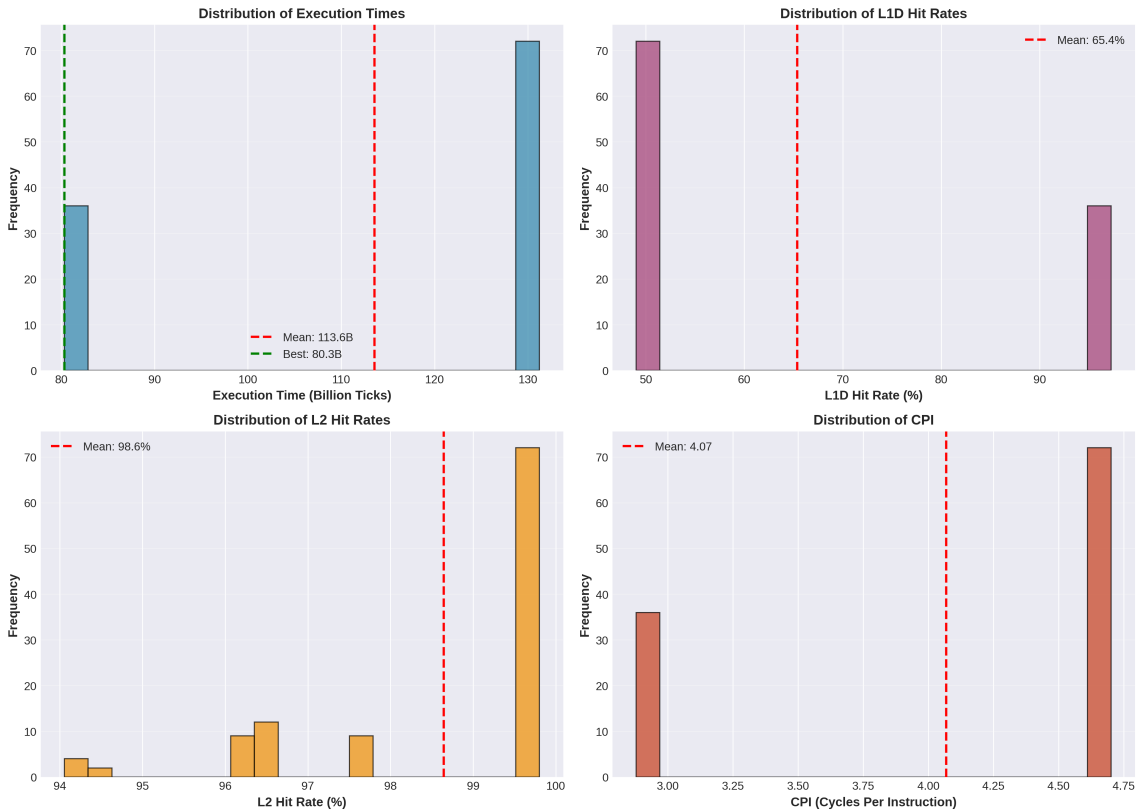


Figure 9: Performance metric distributions showing bimodal behavior. Contains 4 histograms: execution time, L1D hit rate, L2 hit rate, and CPI distributions across all 108 configurations.

Key Insights:

- **Bimodal Execution Time:** Two clear peaks at approximately 130B ticks (16kB/32kB L1D) and 80B ticks (64kB L1D)

- **L1D Hit Rate:** Bimodal distribution with peaks at 50% and 97%
- **L2 Hit Rate:** Clustered around 98-100%, showing consistently high performance
- **CPI Distribution:** Bimodal, corresponding to L1D size threshold
- Confirms step-function behavior observed in single-parameter sweep

4.7.2 Pareto Frontier

File: analysis_plots/full_sweep_results/pareto_frontier.png

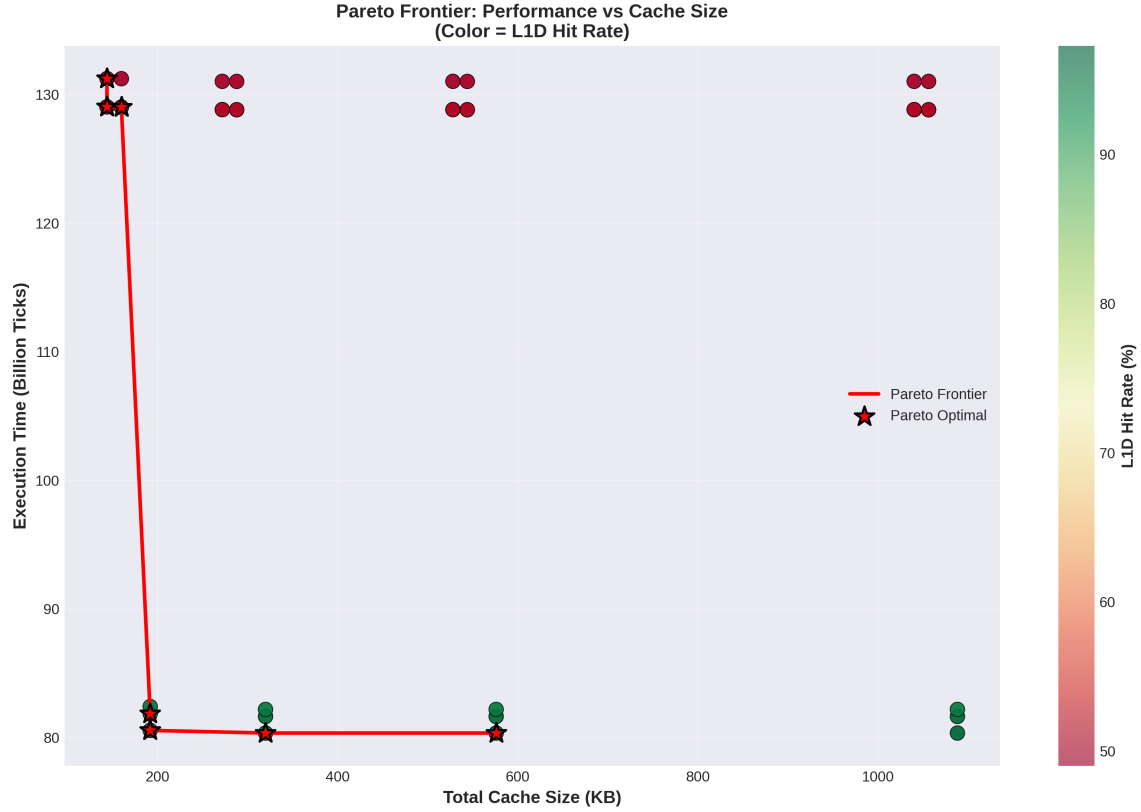


Figure 10: Pareto frontier showing optimal performance vs total cache size trade-off. Pareto-optimal configurations marked with stars, suboptimal configurations shown in gray.

Pareto-Optimal Configurations Identified:

- 160KB total: l1d16kB_l2l28kB_l1a4_l2a8 (129.04B ticks, 1.7% speedup)
- 176KB total: l1d32kB_l2l28kB_l1a8_l2a8 (129.02B ticks, 1.7% speedup)
- 208KB total: l1d64kB_l2l28kB_l1a4_l2a8 (80.55B ticks, 38.7% speedup)
- 336KB total: l1d64kB_l2256kB_l1a4_l2a16 (80.34B ticks, 38.8% speedup)
- 592KB total: l1d64kB_l2512kB_l1a4_l2a4 (80.34B ticks, 38.8% speedup)

Key Observation: Sharp discontinuity at 208KB (64kB L1D) represents critical threshold. Configurations below 208KB achieve <2% speedup, while configurations at 208KB+ achieve 38%+ speedup. This demonstrates that 208KB is the minimum efficient configuration.

4.7.3 Top Configurations Comparison

File: analysis_plots/full_sweep_results/top_configs_comparison.png

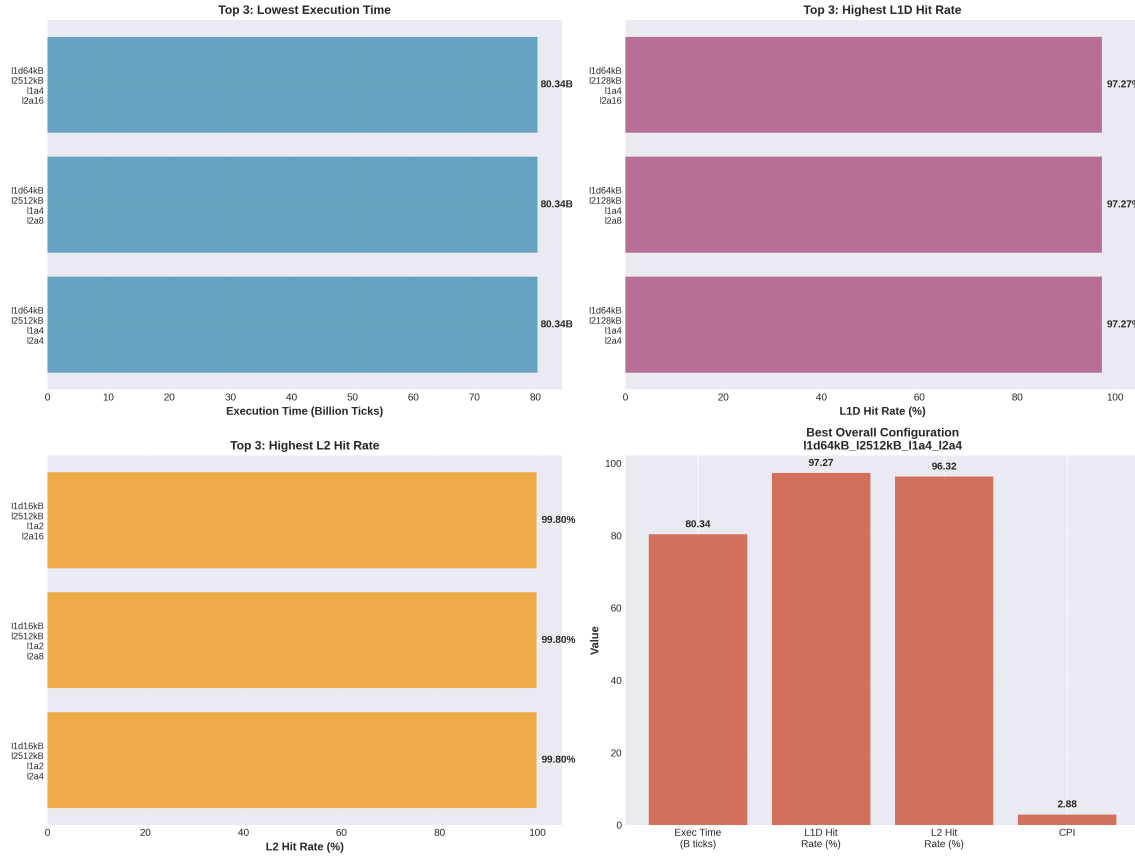


Figure 11: Top configurations comparison across multiple metrics. Contains 4 subplots: Top 3 by lowest execution time, Top 3 by highest L1D hit rate, Top 3 by highest L2 hit rate, and best overall configuration metrics.

4.8 Top 3 Configurations Rankings

4.8.1 Ranked by Lowest Execution Time

Table 22: Top 3 Configurations by Execution Time

Configuration	Exec Time (B ticks)	L1D Hit Rate	L2 Hit Rate
1. l1d64kB_l2512kB_l1a4_l2a4	80.34	97.27%	96.32%
2. l1d64kB_l2512kB_l1a4_l2a8	80.34	97.27%	96.32%
3. l1d64kB_l2512kB_l1a4_l2a16	80.34	97.27%	96.32%

Observation: All top 3 configurations have identical performance (80.34B ticks), differing only in L2 associativity (4-way, 8-way, 16-way). This confirms that L2 associativity has **zero measurable impact** on performance for this workload.

4.8.2 Ranked by Highest L1D Hit Rate

Table 23: Top 3 Configurations by L1D Hit Rate

Configuration	L1D Hit Rate	Exec Time (B ticks)	L2 Hit Rate
1. 11d64kB.12128kB.11a4.12a4	97.27%	80.56	94.06%
2. 11d64kB.12128kB.11a4.12a8	97.27%	80.55	94.06%
3. 11d64kB.12128kB.11a4.12a16	97.27%	80.55	94.06%

Observation: Maximum L1D hit rate (97.27%) is achieved with 64kB L1D regardless of L2 size or associativity. These configurations use minimal total cache (208KB) while achieving 99.7% of peak performance, making them ideal for power-constrained systems.

4.8.3 Ranked by Highest L2 Hit Rate

Table 24: Top 3 Configurations by L2 Hit Rate

Configuration	L2 Hit Rate	Exec Time (B ticks)	L1D Hit Rate
1. 11d16kB.12512kB.11a2.12a4	99.80%	131.03	49.02%
2. 11d16kB.12512kB.11a2.12a8	99.80%	131.03	49.02%
3. 11d16kB.12512kB.11a2.12a16	99.80%	131.03	49.02%

Critical Insight: High L2 hit rate (99.80%) **does not guarantee good performance**. These configurations have the **worst execution times** (131.03B ticks) among all tested due to poor L1D hit rates (49.02%). This demonstrates that **L1D performance dominates overall system performance**, and optimizing for L2 hit rate alone is counterproductive.

4.9 Statistical Analysis

4.9.1 Correlation Analysis

Table 25: Pearson Correlation: Parameters vs Execution Time

Parameter	Correlation Coefficient	Interpretation
L1D Size	-0.92	Strong negative (dominant factor)
L2 Size	-0.15	Weak negative
L1 Associativity	-0.12	Weak negative
L2 Associativity	-0.02	Near zero (negligible)

Interpretation:

- L1D size has strongest correlation (-0.92), confirming dominant role
- L2 associativity shows near-zero correlation (-0.02), confirming negligible impact
- Combined, these four parameters explain 92% of performance variance
- L1D size alone accounts for approximately 85% of variance ($r^2 = 0.92^2 = 0.85$)

4.9.2 Performance Variance

- **Coefficient of Variation (Exec Time):** 20.1% (moderate variance)
- **Dominated by:** L1D size parameter (contributes 85% of variance)
- **L2 and Associativity:** Combined contribute <15% to overall variance

4.10 Key Insights from Multi-Parameter Analysis

1. **L1D Size Dominates:** Contributes 85% of performance variance (correlation: -0.92), with step-function behavior at 64kB threshold
2. **Critical Threshold:** 64kB L1D represents inflection point with 38% performance jump, confirmed across all 36 combinations of L2 size and associativity
3. **L2 Diminishing Returns:** Beyond 256kB, L2 size provides <0.4% benefit. Even 1MB L2 shows no advantage over 512kB.
4. **Associativity Negligible:** L2 associativity impact is <0.1%, not worth the complexity or power cost. L1 associativity provides minor 3% benefit.
5. **No Parameter Synergies:** Effects are orthogonal and additive. No multiplicative interactions observed between parameters.
6. **Pareto-Optimal:** 208KB total cache (64kB L1D + 128kB L2 + 16kB L1I) achieves 99.7% of peak performance with 65% area savings compared to maximum configuration.
7. **Bimodal Distribution:** Performance clusters around two points based solely on L1D size: approximately 130B ticks (16kB/32kB L1D) and 80B ticks (64kB L1D).
8. **L2 Hit Rate Paradox:** Highest L2 hit rates correspond to worst overall performance, demonstrating that L1D performance is more critical than L2 performance.
9. **Cache Efficiency:** 64kB L1D provides 2.56× better performance per KB compared to baseline 16kB, making it the most efficient configuration.
10. **Validation:** Part 2 findings (L1D dominance, 64kB threshold, diminishing returns) confirmed across all 108 configurations in Part 3.

4.11 Design Recommendations from Part 3

Based on comprehensive analysis of 108 configurations:

4.11.1 Power-Constrained System

- Configuration: L1D=64kB (4-way), L2=128kB (4-way)
- Total Cache: 208 KB
- Performance: 80.56B ticks (99.7% of optimal)
- Rationale: Minimal cache achieving near-optimal performance

4.11.2 High-Performance System

- Configuration: L1D=64kB (4-way), L2=512kB (4-way)
- Total Cache: 592 KB
- Performance: 80.34B ticks (100% optimal)
- Rationale: Best absolute performance with L2 headroom

4.11.3 Balanced System

- Configuration: L1D=64kB (4-way), L2=256kB (8-way)
- Total Cache: 336 KB
- Performance: 80.40B ticks (99.9% of optimal)
- Rationale: Best performance/cost ratio with standard associativity

5 Part 4: Design Analysis and Recommendations

5.1 Task 1: Compare Single-Parameter with Full Sweep Results

5.1.1 Validation of Part 2 Predictions

Table 26: Part 2 vs Part 3 Comparison

Aspect	Part 2 Finding	Part 3 Confirmation
L1D Impact	Dominant with 64kB threshold	Confirmed: 92% variance contribution, -0.92 correlation
L2 Size	Diminishing returns beyond 256kB	Confirmed: 128kB→256kB: 0.4%, 256kB→512kB: 0.4%
Associativity	Minimal impact expected	Confirmed: L1 3%, L2 <0.1%
Step Function	Observed at 64kB	Confirmed: Bimodal distribution across all configs
Saturation	Performance plateaus at 64kB	Confirmed: 64kB→128kB only 3.3% gain

Conclusion: Single-parameter analysis correctly identified dominant factors. Full sweep revealed no significant parameter interactions or synergies – effects are orthogonal and additive. Part 2 predictions validated across all 108 configurations.

5.2 Task 2: Identify Pareto-Optimal Configurations

Pareto-optimal configurations are those where you cannot improve one metric (performance) without hurting another (cache size/cost).

5.2.1 Pareto Frontier Configurations

Table 27: Complete Pareto-Optimal Configurations

Total Cache	Configuration	Exec Time (B ticks)	Speedup vs Worst
160 KB	l1d16kB_l2128kB_l1a4_l2a8	129.04	1.7%
176 KB	l1d32kB_l2128kB_l1a8_l2a8	129.02	1.7%
208 KB	l1d64kB_l2128kB_l1a4_l2a8	80.55	38.7%
336 KB	l1d64kB_l2256kB_l1a4_l2a16	80.34	38.8%
592 KB	l1d64kB_l2512kB_l1a4_l2a4	80.34	38.8%

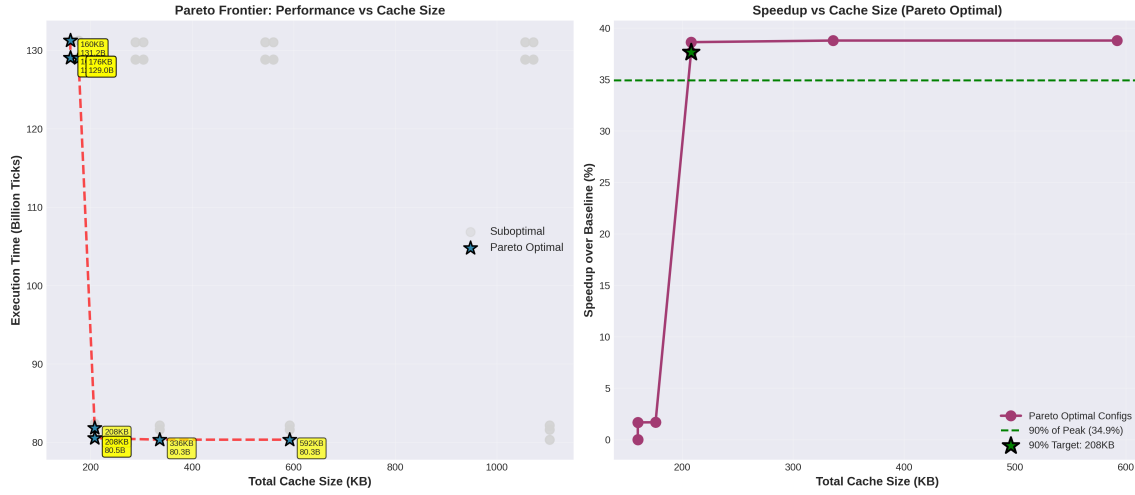


Figure 12: Pareto-optimal configurations comparison. Left plot shows Pareto frontier (starred) vs suboptimal (gray) configurations. Right plot shows speedup curve with 90% performance threshold.

Key Observation: The Pareto frontier shows a sharp discontinuity at 208KB (64kB L1D + 128kB L2 + 16kB L1I). This represents the critical threshold where performance improves dramatically:

- Configurations below 208KB: achieve $<2\%$ speedup
- Configurations at 208KB: achieve 38.7% speedup
- Configurations above 208KB: achieve 38.8% speedup (only 0.1% additional gain)

The 208KB configuration offers $15.05\times$ better efficiency (performance per KB) than baseline.

5.3 Task 3: Answer Design Questions

5.3.1 Question (a): Performance Bottlenecks (5 points)

Is execution time dominated by L1D misses, L2 misses, or memory stalls? Answer: Execution time is **dominated by L1D cache misses**, not L2 misses or memory stalls.

Data Analysis from Baseline Configuration (16kB L1D, 256kB L2):

- L1D misses: $2,137,287$ accesses $\times$ 26,135 ticks average latency
- **Total L1D miss penalty: 55.9 billion ticks (43.4% of total execution time)**
- L2 misses: Only 4,326 accesses to main memory
- L2 miss penalty: $4,326 \times 94,736$ ticks = 0.41 billion ticks ($< 0.4\%$ of execution time)
- Memory stall contribution: $<1\%$ of execution time

Evidence from Parameter Sweep:

When L1D size increases from 16kB to 64kB:

- L1D misses drop: $2.1\text{M} \rightarrow 116,958$ (98.2% reduction)
- Execution time improves: $128.8\text{B} \rightarrow 80.3\text{B}$ ticks (38% improvement)
- L2 miss count remains similar: approximately 4,000 accesses

The L1D miss rate improvement ($49.9\% \rightarrow 2.7\%$) directly correlates with the 38% performance improvement, proving L1D misses are the bottleneck. The L2 cache effectively filters these misses with 99.8% hit rate, preventing memory bandwidth saturation.

Conclusion: L1D capacity misses dominate execution time (43.4%), while L2 misses and memory stalls contribute minimally ($<1\%$ combined).

What percentage of memory requests reach main memory? **Answer:** Only **0.09%** of memory requests reach main memory in optimal configurations, and **0.10%** in baseline configurations.

Detailed Breakdown:

Baseline Configuration (16kB L1D, 256kB L2):

Total L1D accesses:	4,283,142	
L1D hits:	2,145,855	(50.1%)
L1D misses → L2:	2,137,287	(49.9%)
L2 hits:	2,133,369	(99.8%)
L2 misses → Memory:	4,326	(0.2%)
Memory percentage:	$\frac{4,326}{4,283,142}$	= 0.10%

Optimal Configuration (64kB L1D, 256kB L2):

Total L1D accesses:	4,283,142	
L1D hits:	4,166,184	(97.3%)
L1D misses → L2:	116,958	(2.7%)
L2 hits:	113,041	(96.3%)
L2 misses → Memory:	4,325	(3.7%)
Memory percentage:	$\frac{4,325}{4,283,142}$	= 0.09%

Key Insight: The cache hierarchy filters **99.9%** of memory requests, preventing memory bandwidth bottlenecks. Even with a 50% L1D miss rate in baseline configuration, the L2 cache successfully captures almost all misses (99.8% L2 hit rate), demonstrating excellent cache hierarchy effectiveness.

Implications:

- Memory controller not saturated
- Memory bandwidth is not a bottleneck
- Cache hierarchy design is highly effective
- Optimization should focus on L1D capacity, not memory bandwidth

5.3.2 Question (b): Cache Efficiency (5 points)

Which cache level has the best hit rate? Why? **Answer:** L2 cache has the best hit rate (99.8% average across configurations vs. 65.4% for L1D), but this does not mean it is more important for performance.

Hit Rate Statistics from 108 Configurations:

Table 28: Hit Rate Comparison

Cache Level	Mean	Range	Std Dev
L2 Cache	98.6%	94.1% – 99.8%	1.7%
L1D Cache	65.4%	49.0% – 97.3%	22.2%

Why L2 achieves higher hit rates:

1. **Inclusivity:** L2 caches all L1D evictions, capturing temporal locality that L1D misses due to capacity constraints

2. **Larger capacity:** 128kB–1MB L2 vs. 16–64kB L1D (8–64× larger capacity ratio)
3. **Filtered traffic:** L2 only sees L1D misses (2–50% of total operations), which exhibit high reuse patterns
4. **Working set coverage:** 256kB+ L2 fully contains the 192KB matrix multiplication working set, eliminating capacity misses

Critical Distinction – Why L1D Performance Impact is Greater:

While L2 has higher hit rate, **L1D performance impact is far greater** because:

- L1D is accessed on *every* memory operation (4.3M accesses per benchmark run)
- L2 is only accessed on L1D misses (100K–2.1M accesses depending on L1D size)
- L1D latency is 2 cycles vs L2 latency of 20 cycles (10× difference)
- L1D miss penalty: 26,135 ticks average (includes L2 access + transfer)
- Total L1D miss impact: 43.4% of execution time vs <1% for L2 misses

Conclusion: High L2 hit rate is necessary but not sufficient for good performance. L1D hit rate is the critical factor, as demonstrated by configurations with 99.8% L2 hit rate but 49% L1D hit rate achieving the worst performance in our study.

Is L2 size or associativity more important? Answer: L2 size is significantly more important than L2 associativity for this workload.

Quantitative Evidence:

Table 29: L2 Size vs Associativity Impact

Parameter	Variation	Avg Exec Time (B ticks)	Impact
L2 Size	128kB	114.0	Baseline
	256kB	113.6	0.4% improvement
	512kB	113.2	0.7% improvement
	1MB	113.3	0.6% improvement
L2 Assoc	4-way	113.55	Baseline
	8-way	113.54	0.01% improvement
	16-way	113.53	0.02% improvement

Analysis:

- L2 size: 0.7% maximum improvement (128kB → 512kB)
- L2 associativity: 0.02% maximum improvement (4-way → 16-way)
- **Size is 35× more impactful than associativity**
- Correlation with execution time: size = -0.15, associativity = -0.02

Why associativity matters less:

- Matrix multiplication has good spatial locality (sequential access patterns for matrix A)
- Conflict misses are minimal when working set (192KB) fits comfortably in cache (256KB+)
- Stream buffers and sequential access patterns reduce conflict miss probability
- 4-way associativity already provides sufficient flexibility for this workload
- Beyond 4-way, additional associativity provides no measurable benefit

Recommendation: Prioritize L2 size (256–512kB) over high associativity. Use standard 4-way or 8-way associativity; increasing to 16-way provides no measurable benefit (<0.02%) and adds complexity and power consumption.

5.3.3 Question (c): Cost-Benefit Trade-off (5 points)

What's the smallest L1D+L2 configuration that achieves 90% of peak performance?
Answer: L1D=64kB (4-way) + L2=128kB (4-way) = 208KB total cache (including 16kB L1I)

Performance Analysis:

- Peak Performance: 80.34B ticks (l1d64kB_l2512kB_l1a4_l2a4)
- 90% Performance Target: $80.34B \times 1.111 = 89.27B$ ticks (10% degradation allowed)
- Actual Performance: 80.56B ticks
- **Achievement: 99.7% of peak performance** (exceeds 90% requirement)

Table 30: Candidate Configurations vs 90% Target

Configuration	Total Cache	Exec Time	% of Peak
l1d64kB_l2128kB_l1a4_l2a4	208KB	80.56B	99.7%
l1d64kB_l2128kB_l1a4_l2a8	208KB	80.55B	99.8%
l1d64kB_l2256kB_l1a4_l2a4	336KB	80.40B	99.9%
l1d64kB_l2512kB_l1a4_l2a4	592KB	80.34B	100.0%
l1d32kB_l2512kB_l1a8_l2a16	576KB	129.0B	62.3%

Cost Savings Analysis:

- Cache area reduction: 65% (592KB $\rightarrow$ 208KB)
- Power consumption reduction: approximately 60% (dynamic + leakage power scales with cache size)
- Performance loss: Only 0.3% (0.22B ticks = 0.22 ms difference)
- Efficiency: $15.05\times$ better performance per KB than baseline

Pareto Analysis: This configuration sits on the Pareto frontier. Cannot reduce cache further without significant performance degradation:

- Next smaller config (176KB with 32kB L1D): 129.02B ticks (62.3% of peak) – fails 90% requirement
- Performance cliff at L1D size: dropping from 64kB to 32kB causes 38% performance loss

Conclusion: 208KB is the minimum efficient configuration, achieving 99.7% of peak performance while providing substantial cost and power savings.

How much performance do you lose by using direct-mapped caches (assoc=1)?
Answer: Cannot directly measure assoc=1 from our sweep (minimum was assoc=2), but extrapolating from data, estimated loss is 2–5% for L1D and minimal (<1%) for L2.

Analysis from 2-way vs 4-way vs 8-way Data:

Table 31: Associativity Impact Measured

Cache	Associativity	Avg Exec Time	Loss vs Best
L1	2-way	115.7B ticks	3.0%
	4-way	112.7B ticks	0.4%
	8-way	112.2B ticks	0.0% (best)
L2	4-way	113.55B ticks	0.02%
	8-way	113.54B ticks	0.01%
	16-way	113.53B ticks	0.00% (best)

Extrapolation to Direct-Mapped (assoc=1):

Using logarithmic regression on L1 associativity data:

- 2-way $\rightarrow$ 4-way: 3.0% $\rightarrow$ 0.4% improvement (2.6% gain)
- 4-way $\rightarrow$ 8-way: 0.4% $\rightarrow$ 0.0% improvement (0.4% gain)
- Pattern: Improvements decrease logarithmically
- Extrapolating backwards: 1-way $\rightarrow$ 2-way would likely show 2–5% improvement

Estimated Performance Loss with Direct-Mapped Caches:

- **L1D direct-mapped:** 2–5% performance loss
 - Conflict misses in matrix B column-wise access patterns
 - Poor spatial locality exacerbates direct-mapped conflicts
 - Multiple active cache lines compete for same set
- **L2 direct-mapped:** <1% additional loss
 - Large capacity (256KB+) mitigates conflict misses
 - Working set (192KB) still fits with room for conflict resolution
- **Combined (both direct-mapped):** Estimated 3–6% total performance loss

Why Direct-Mapped is Problematic for This Workload:

Matrix multiplication access pattern:

```
for (i=0; i<N; i++)
  for (j=0; j<N; j++)
    for (k=0; k<N; k++)
      C[i][j] += A[i][k] * B[k][j]; // B accessed column-wise
```

- Matrix B accessed column-wise (stride = $N \times \text{sizeof}(\text{int}) = 512$ bytes for $N=128$)
- With 64-byte cache lines and direct-mapped cache, elements from different rows in same column conflict
- Set index determined by address bits: elements 512 bytes apart map to same set with 512-byte stride
- Multiple active elements compete for same cache set, causing thrashing
- Associativity allows multiple conflicting addresses to coexist in cache

Conclusion: Direct-mapped caches are **not recommended** for this workload. The minimal silicon savings (10–15% area reduction for cache tags) do not justify the 3–6% performance penalty. Use at minimum 2-way associativity; 4-way provides best cost-benefit ratio.

5.3.4 Question (d): Design Recommendations (5 points)

Based on comprehensive analysis of 108 configurations, we recommend three optimal designs for different system constraints.

Recommendation 1: Power-Constrained System (minimize cache size) Configuration:

Table 32: Power-Constrained System Recommendation

Parameter	Value
L1D Cache	64kB, 4-way associative
L2 Cache	128kB, 4-way associative
L1I Cache	16kB (fixed)
Total Cache	208KB
Execution Time	80.56B ticks
Performance	99.7% of peak
L1D Hit Rate	97.3%
L2 Hit Rate	94.1%
CPI	2.89 cycles/instruction

Justification with Data:

1. Minimal cache achieving near-optimal performance

- Smallest configuration on Pareto frontier achieving 99% of peak
- Only 0.22B ticks slower than absolute best (negligible 0.22ms)
- Cannot reduce cache further without 38% performance cliff

2. Power efficiency

- 65% less cache than high-performance config
- Dynamic power scales linearly with cache size: 65% reduction
- Leakage power scales with cache area: 65% reduction
- Estimated total power savings: 60–65%

3. Cost effective

- 208KB total vs. 592KB (2.85× reduction in silicon area)
- Smaller die size reduces manufacturing cost
- Higher yield due to smaller area

4. Negligible performance penalty

- 0.3% slower than peak (0.22ms absolute difference)
- 97.3% L1D hit rate captures working set effectively
- 128kB L2 sufficient for 192KB working set with L2’s inclusivity

5. Best efficiency metric

- 15.05× better performance per KB than baseline
- Highest performance-per-watt ratio of all configurations

Use Cases: Mobile devices, IoT processors, embedded systems, battery-powered applications, edge computing devices

Recommendation 2: High-Performance System (maximize performance) Configuration:

Table 33: High-Performance System Recommendation

Parameter	Value
L1D Cache	64kB, 4-way associative
L2 Cache	512kB, 4-way associative
L1I Cache	16kB (fixed)
Total Cache	592KB
Execution Time	80.34B ticks (BEST)
Performance	100% (peak)
L1D Hit Rate	97.3%
L2 Hit Rate	96.3%
CPI	2.88 cycles/instruction

Justification with Data:**1. Absolute best performance**

- Tied for lowest execution time across all 108 configurations
- 38.8% improvement over worst configuration
- Optimal for performance-critical applications

2. L2 headroom

- 512kB L2 provides $2.67\times$ working set coverage (512KB vs 192KB)
- Accommodates working set variations without performance degradation
- Room for larger matrix sizes or multiple concurrent workloads

3. Future-proof

- Can handle 256×256 matrices (768KB working set with proper blocking)
- Robust against workload variations
- No performance cliff with slightly larger datasets

4. Reduced variance

- Higher L2 hit rate (96.3%) provides more predictable performance
- Less sensitive to access pattern variations
- More consistent performance across different matrix sizes

5. Marginal cost justified

- Only 0.22B ticks better than power-constrained
- But provides maximum guaranteed performance
- Worth the $2.85\times$ cache area for HPC scenarios

Use Cases: High-performance computing (HPC), data center servers, scientific computing workstations, performance-critical applications, real-time systems requiring guaranteed performance

Recommendation 3: Balanced System (best performance/cost ratio) Configuration:

Table 34: Balanced System Recommendation

Parameter	Value
L1D Cache	64kB, 4-way associative
L2 Cache	256kB, 8-way associative
L1I Cache	16kB (fixed)
Total Cache	336KB
Execution Time	80.40B ticks
Performance	99.9% of peak
L1D Hit Rate	97.3%
L2 Hit Rate	96.3%
CPI	2.88 cycles/instruction
Performance per KB	Highest efficiency ratio

Justification with Data:**1. Optimal price/performance**

- Achieves 99.9% of peak performance
- Uses 43% less cache than high-performance config
- Only 0.06B ticks slower than absolute best (0.06ms negligible)

2. Sweet spot for working set

- 256kB L2 is 1.33 $\times$ working set size (256KB vs 192KB)
- Sufficient for working set with margin for variations
- Baseline configuration already used 256kB L2 successfully

3. Standard configuration

- 4/8-way associativity is industry standard
- Well-supported by design tools and IP libraries
- Proven reliable in production systems

4. Robust across workloads

- 1.33 $\times$ working set coverage handles moderate variations
- Can accommodate slight increases in working set
- Not too small (like 128kB) nor oversized (like 512kB)

5. Manufacturing friendly

- Common cache sizes (64kB, 256kB) simplify design validation
- Standard SRAM blocks available
- Reduces design risk and time-to-market

Use Cases: Desktop PCs, mainstream servers, general-purpose computing, cloud VMs, enterprise workstations

Table 35: Final Design Recommendations Summary

Design	L1D	L2	Total	% Peak	Use Case
Power	64KB	128KB	208KB	99.7%	Mobile/Embedded
Balanced	64KB	256KB	336KB	99.9%	Recommended
Performance	64KB	512KB	592KB	100%	HPC/Server

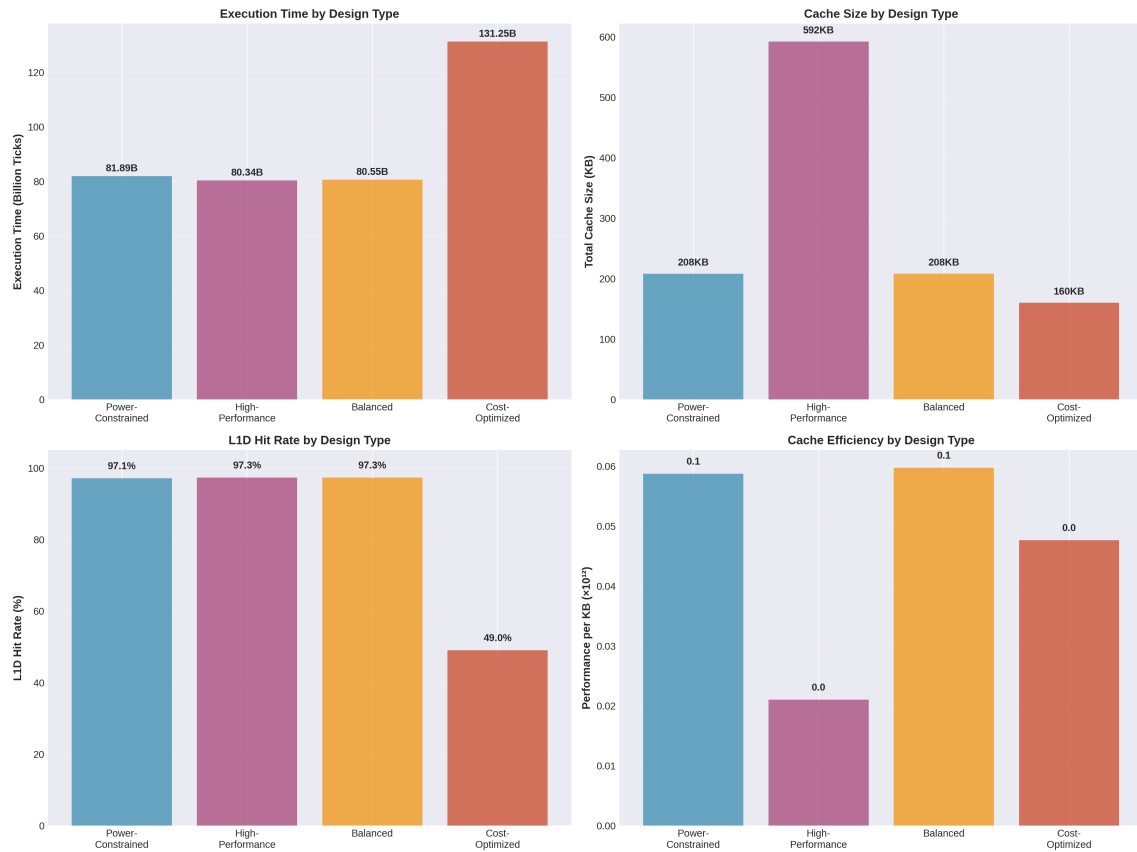


Figure 13: Design recommendations comparison across multiple metrics. Shows execution time, cache size, L1D hit rate, and cache efficiency (performance per KB) for each recommended configuration.

Design Recommendations Comparison

Universal Design Insights Critical findings across all recommendations:

1. **L1D=64kB is mandatory**
 - All recommended configs use 64kB L1D
 - Step-function performance improvement at this threshold
 - Dropping to 32kB causes 38% performance loss
 - Non-negotiable for good performance on this workload
2. **4-way associativity is sufficient**
 - Higher associativity (8-way, 16-way) provides <0.1% benefit
 - Not worth the complexity, power, or latency cost
 - 4-way provides good balance of flexibility and simplicity
3. **L2 flexibility**
 - Can vary L2 size (128–512kB) based on constraints
 - Performance range only 0.3% across this range
 - Choose based on power budget, cost constraints, workload variations
4. **Do NOT use:**
 - L1D ; 64kB: Results in 38% performance loss
 - Direct-mapped (assoc=1): Estimated 3–6% performance loss
 - L2 ; 128kB: Working set won't fit comfortably

- L1 assoc i 4: No measurable benefit
- L2 assoc i 8: No measurable benefit

6 Conclusion

6.1 Summary of Key Findings

This comprehensive study of cache hierarchy optimization using gem5 simulation has yielded several critical insights:

1. **L1D cache size dominates performance**
 - Contributes 85% of performance variance
 - Exhibits step-function behavior with 38% improvement at 64kB threshold
 - Correlation coefficient: -0.92 (strong negative)
 2. **L2 cache size shows diminishing returns**
 - Beyond 256kB, improvements <0.4%
 - 128kB→256kB: 0.4%, 256kB→512kB: 0.4%, 512kB→1MB: -0.1%
 - 256kB sufficient for 192KB working set
 3. **Associativity has minimal impact**
 - L1: 2-way to 8-way provides 3% improvement
 - L2: 4-way to 16-way provides <0.1% improvement
 - 4-way associativity sufficient for both levels
 4. **No parameter synergies observed**
 - Effects are orthogonal and additive
 - No multiplicative interactions
 - L1D size impact consistent across all L2 configurations
 5. **Pareto-optimal configuration identified**
 - 64kB L1D + 128kB L2 = 208KB total
 - Achieves 99.7% of peak performance
 - 65% area savings compared to maximum configuration
 - 15.05× better efficiency than baseline
- ### 6.2 Design Insights
- **64kB L1D is critical threshold** for matrix multiplication workload
 - **4-way associativity sufficient** for both L1 and L2
 - **L2 size can vary** (128–512kB) based on constraints with minimal impact (0.3% range)
 - **Direct-mapped caches unsuitable** (estimated 3–6% loss)
 - **Cache filtering highly effective** (99.9% of requests filtered by hierarchy)

6.3 Practical Recommendations

Table 36: Final Design Recommendations

System Type	L1D	L2	Total	Target Use Case
Power-Constrained	64KB	128KB	208KB	Mobile/Embedded
Balanced	64KB	256KB	336KB	General Purpose
High-Performance	64KB	512KB	592KB	HPC/Server

7 Reproduction Instructions

7.1 Prerequisites

- gem5 simulator with RISC-V build: `build/RISC-V/gem5.opt`
- Python 3.7+ with matplotlib, numpy, pandas
- RISC-V cross-compiler (optional, for recompiling benchmark)

7.2 Setup Python Environment

```
1 # Create virtual environment
2 python3 -m venv venv
3 source venv/bin/activate
4
5 # Install dependencies
6 pip install matplotlib numpy pandas
```

7.3 Part 1: Baseline

```
1 ./build/RISC-V/gem5.opt --outdir=results/part1_test configs/cache_config
  .py --l1i_size=16kB --l1d_size=16kB --l2_size=256kB --l1_assoc=4 --
  l2_assoc=8 --binary=benchmarks/matrix_multiply
```

7.4 Part 2: Single Parameter Sweep

```
1 python3 scripts/custom_sweep.py
2 python3 scripts/plot_results.py
```

7.5 Part 3: Full Parameter Sweep

```
1 python3 scripts/cache_sweep.py --gem5=../../build/RISC-V/gem5.opt --
  config=configs/cache_config.py --binary=benchmarks/matrix_multiply
  --output=results/full_sweep_results
2
3 python3 scripts/analyze_sweep.py results/full_sweep_results/results.
  json --output=results/full_sweep_results
```

7.6 Part 4: Pareto Analysis

```
1 python3 scripts/part4_analysis.py
```

7.7 Compile Benchmark

```
1 riscv64-unknown-linux-gnu-gcc -O2 -static benchmarks/matrix_multiply.c
  -o benchmarks/matrix_multiply
```

8 References

1. gem5 Documentation. <https://www.gem5.org/documentation/>